



Digital Egypt Pioneers Initiative
Penetration Testing Track
ALX3_ISS3_S1 Group

Graduation Project Documentation

Secure Medical Appointment System

Presented by

Omar Magdy Abdallah ElKhazendar	21001646
Kerelos Zakarya Tadros Botros	21033693
Ziad Mahmoud Ahmed Abdelgwad	21065393
Ahmed Mourad Mohamed Anwer	21007626
Ahmed Mohamed Ashri Mourad	21049580
Abdelrahman Ashraf Mohamed Abdo	21117329

Supervised by:
Eng. Ziead Ebrahim

Round 3

1. Project Planning

Purpose. Build two functionally equivalent medical systems: one intentionally secure, one intentionally insecure, and perform controlled penetration testing on the insecure system using the tools you named (sqlmap, nmap, XSS testing, Burp Suite). The documentation produced here supports development, testing, and submission.

Scope.

- Implement core user roles and features: admin, doctor, patient; appointment management; medical records; file uploads; basic social interactions where present.
- Maintain two branches/environments: a secure branch and an insecure lab branch for pentesting.
- Use only authorized, isolated test environments for any security testing.

Deliverables.

1. Two deployed application instances (secure / insecure).
2. Project documentation covering planning, stakeholders, database design, and UI/UX.
3. Penetration test reports and evidence from the insecure system.

High-level phases.

- Requirements & rules of engagement (authorization for pentest).
- Database design and schema review (based on provided SQL).
- Implementation of both systems.
- Deployment: secure instance and isolated insecure lab.
- Penetration testing of insecure instance.
- Documentation and submission.

Risk controls (project-level).

- Run tests only in the isolated lab environment.
 - Preserve database snapshots before testing.
 - Use synthetic/test data in the lab.
-

2. Stakeholder Analysis

This analysis lists the actors who are directly involved in the project and their core responsibilities.

Primary stakeholders

- Project Owner / Instructor — approves the project deliverables and the pentest rules of engagement.
- Development Team — implements both secure and insecure systems, and documents the work.
- Penetration Test Team — executes testing on the insecure system with the agreed tools and documents findings.
- System Administrator — prepares and isolates the test environment; manages backups and snapshots.

Secondary stakeholders

- End-user representatives (doctors / patients) — provide usability feedback for UI/UX evaluation.
- Report recipients / graders — evaluate the final documentation and test evidence.

Responsibilities summary

- Project Owner: sign off on scope and rules of engagement.
- Development Team: deliver application functionality and maintain code branches for secure/insecure builds.
- Pentest Team: perform non-destructive testing within authorization; record evidence.
- SysAdmin: maintain isolation, snapshots, and restore capability.

Acceptance criteria (stakeholder-oriented)

- Functionality present for admin, doctor, and patient roles.
 - Insecure instance accessible only in the lab and clearly documented as test-only.
 - Pentest findings documented and delivered to the Project Owner.
-

3. Database Design

This section describes the logical structure used by the project

Core entities (logical overview).

- **users** — stores accounts used by the application (roles: admin, doctor, patient).
- **patients** — patient-specific profile data linked to users.
- **doctors** — doctor-specific profile data linked to users.
- **appointments** — scheduling information linking patients and doctors.
- **medical_records** — clinical entries associated with patients and authored by doctors.
- **patient_files** — uploaded documents or files associated with patients.
- **activity_logs / audit entries** — application events and user actions.

Relationships (summary).

- Each patient and each doctor is associated with a user account.
- Appointments link a patient to a doctor.
- Medical records reference patient and doctor identifiers.
- Files are associated with patient records or user accounts as appropriate.

Integrity and constraints.

- The schema in the provided SQL contains primary keys and foreign key relationships that enforce referential integrity between users, patients, doctors, appointments, and records. Use the provided CREATE TABLE definitions to validate constraints in the deployed DB.

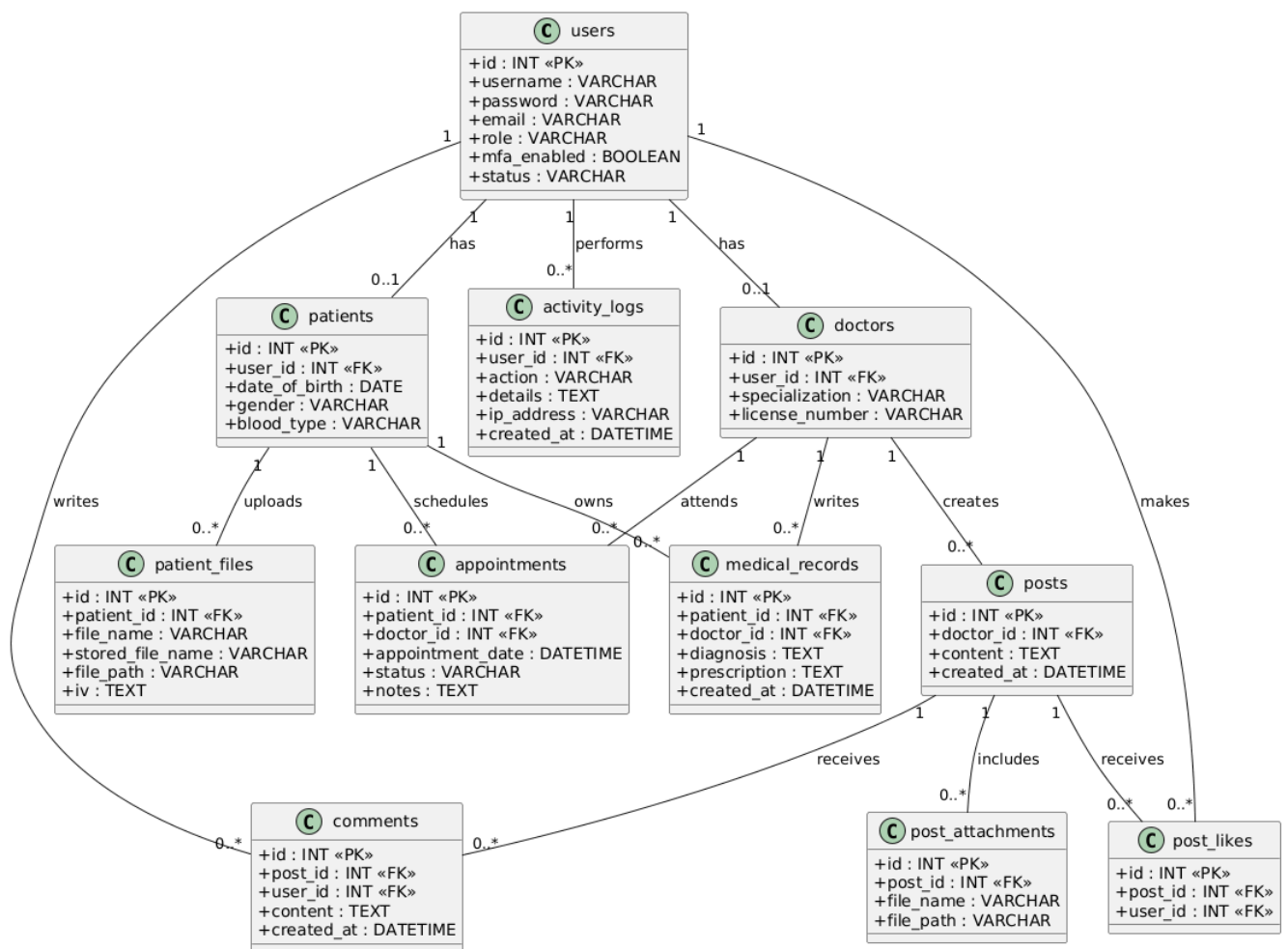
Data handling notes (based on the provided schema).

- The database contains user accounts, medical records, appointment data, and file metadata. Treat the data in the schema as sensitive test data within the lab. Preserve DB snapshots before any testing activity.

Operational conventions (as derived from the schema).

- Index frequently queried fields where present in the SQL.
 - Keep a dedicated backup/snapshot before tests.
 - When performing tests, reference the SQL file to identify table and column names that are in scope.
-

Class Diagram

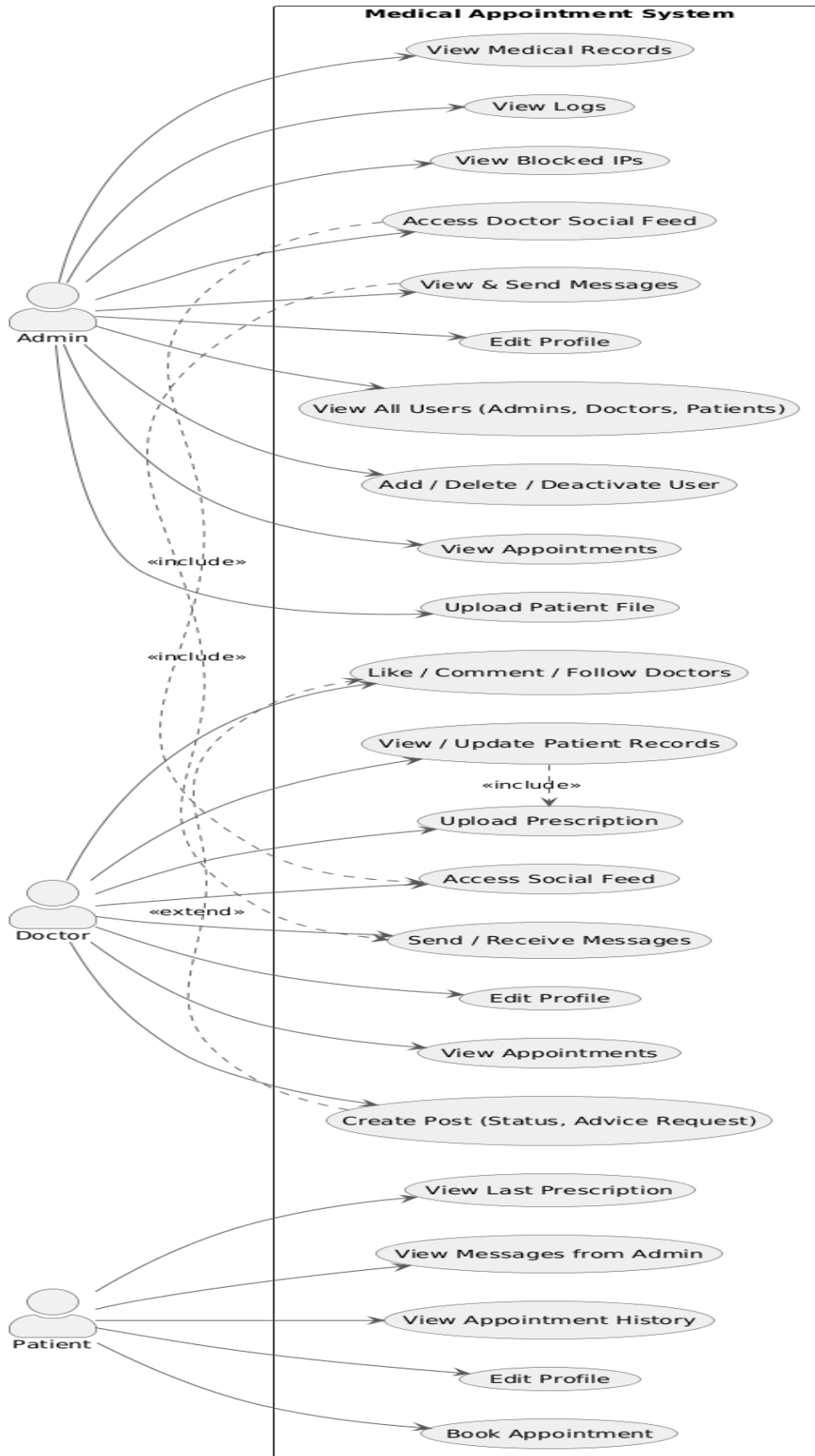


The class diagram represents the main structural components of the medical system based on the provided database schema. It includes the core entities used in the project such as Users, Patients, Doctors, Appointments, Medical Records, Patient Files, Activity Logs, Posts, Post Attachments, Post Likes, and Comments. Each entity is shown as a class containing its key attributes.

The relationships illustrate how information flows between users and medical operations. For example, a patient and doctor are both linked to a user account. Patients and doctors connect through appointments and medical records, while patients can upload documents stored in patient files. System actions performed by users are recorded in activity logs. Additionally, doctors can create posts, which allow attachments, likes, and comments from users.

This diagram supports a structured and clear understanding of the data design, the responsibilities of each entity, and how they interact to serve core healthcare workflows in the project.

Use Case Diagram



4. UI / UX Design

This section describes the user interface roles and the essential screens and flows; it refers only to functionality known from your project description and the database schema.

Primary user roles and dashboards.

- Patient dashboard: view upcoming appointments, access own medical records, and upload files.
- Doctor dashboard: view assigned patients and appointments, create or update medical records, and attach files to records.
- Admin dashboard: manage users and view system activity.

Key screens and flows.

1. Authentication: login screen that accepts user credentials for admin, doctor, and patient accounts (use credentials defined in the deployed environment).
2. Appointments flow: patient selects or requests an appointment; doctor views and manages assigned appointments.
3. Record management: doctor creates or updates a medical record entry linked to a patient; patient can view permitted records.
4. File upload/view: patients or doctors upload files associated with a patient; UI must present file metadata and download/view controls.

Usability and privacy considerations (UI focus only).

- Show only the information appropriate to the current role.
 - Include clear navigation labeled by role-specific sections (Appointments, Records, Files, Messages).
 - Provide confirmation dialogs for sensitive actions (e.g., record deletion).
 - Display audit-visible actions in the UI where relevant (e.g., recent activity) as derived from the schema.
-